Project - High Level Design

on

CI/CD for Go (Golang) Logistics App

Course Name: DevOps

Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	AKSHAT BAJAJ	EN22CS301088
2	AKANKASHA PARIHAR	EN22CS301079
3	AJAY PAL SINGH THAKUR	EN22CS301077
4	AKSHITA FARKYA	EN22CS301099
5	AASHITA CHAVHAN	EN22CS301018
6	AAKRATI JAIN	EN22CS301013

Group Name: Group 05D1

Project Number: DO-05

Industry Mentor Name: Aashruti Shah

University Mentor Name: Prof. Ritesh Joshi

Academic Year: 2025-26

Table of Contents:

1. **Introduction.**
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.
2. **System Design.**
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
3. **Data Design.**
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration
4. **Interfaces**
5. **State and Session Management**
6. **Caching**
7. **Non-Functional Requirements**
   7.1. Security Aspects
   7.2. Performance Aspects
8. **References**

1. Introduction

This document presents the High-Level Design (HLD) for implementing a CI/CD (Continuous Integration and Continuous Deployment) pipeline for a Go-based Logistics Application. The objective of this system is to automate the software development lifecycle, enabling faster delivery, improved reliability, and consistent deployment practices.

The pipeline integrates development, testing, building, and deployment stages into a seamless workflow using modern DevOps tools such as Git, Docker, Jenkins/GitHub Actions, and Kubernetes.

This design ensures:

- Continuous code integration
- Automated testing and validation
- Containerized deployments
- Scalable and fault-tolerant infrastructure.

1.1 Scope of the Document

This document defines:

- Overall architecture of the CI/CD pipeline
- Interaction between different system components
- Application deployment strategy
- Data and process flow across the pipeline
- Security, performance, and scalability considerations

Out of scope:

- Low-Level Design (LLD)
- Exact code implementation
- Tool-specific configuration scripts (like YAML files)

1.2 Intended Audience

This document is intended for:

- DevOps Engineers responsible for pipeline setup
- Developers working on the Go application
- System Architects designing scalable solutions
- Academic evaluators reviewing system design

1.3 System Overview

The system comprises two major parts:

1. Application Layer

A Go-based logistics system handling:

- Order processing
- Shipment tracking
- Delivery updates
- User management

2. CI/CD Pipeline

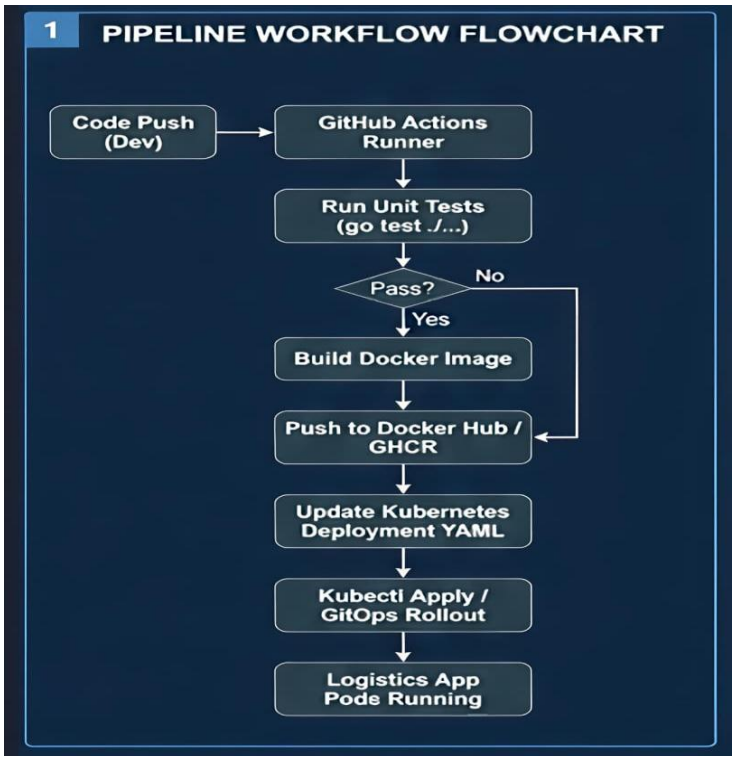

Fig: CI/CD Pipeline

An automated workflow responsible for:

- Code integration and validation
- Build and packaging
- Containerization
- Deployment to Kubernetes

High-Level Workflow

1. Developer pushes code to Git repository
2. CI tool detects changes and triggers pipeline
3. Code is built and tested automatically
4. Docker image is generated
5. Image is pushed to container registry
6. Kubernetes deploys updated application
7. Monitoring tools track system health.

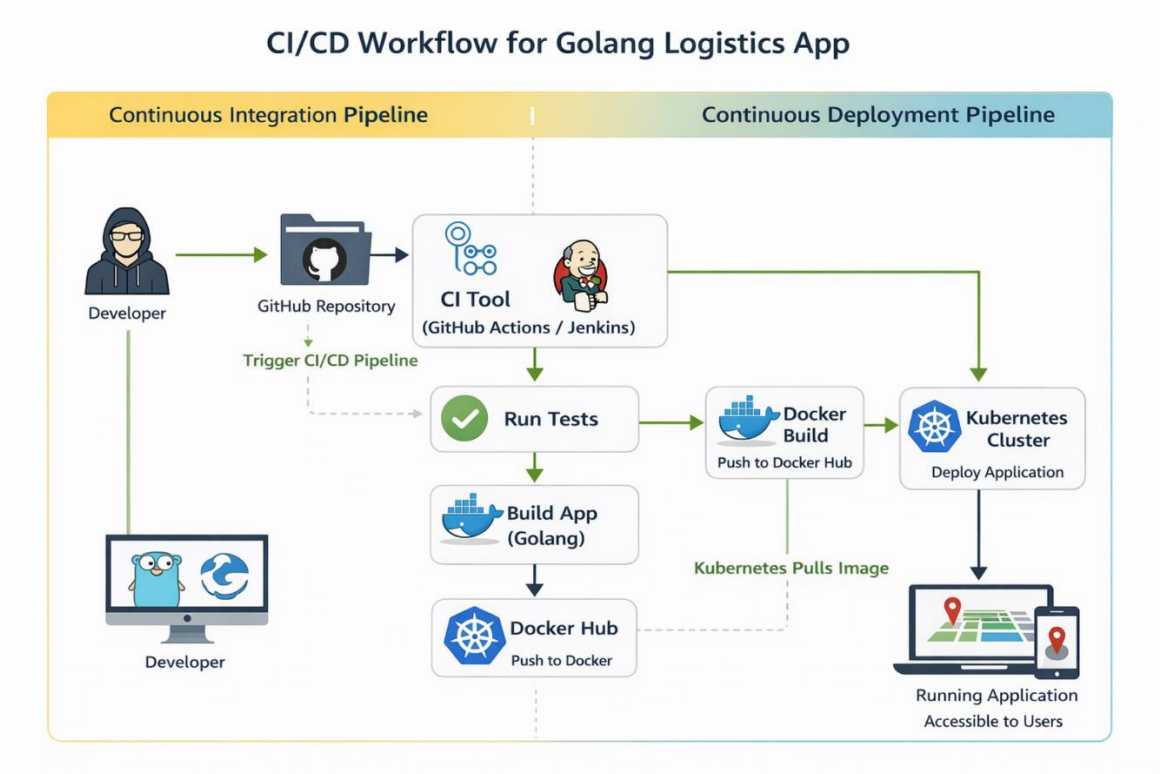

Fig: CI/CD Workflow for Golang Logistics App

2. System Design

2.1 Application Design

The logistics application follows a modular and scalable architecture.

Architecture Style

- RESTful API-based backend
- Microservices (optional but scalable) OR modular monolith

Core Modules

1. **User Management Service**

- Handles authentication and authorization
- Stores user credentials

2. **Order Management Service**

- Creates and manages orders
- Maintains order lifecycle
-

3. **Shipment Service**

- Tracks shipment details
- Updates delivery status

4. **API Gateway**

- Entry point for all client requests
- Routes requests to appropriate services

Deployment Strategy

- Each service runs in a Docker container
- Containers deployed on Kubernetes cluster.

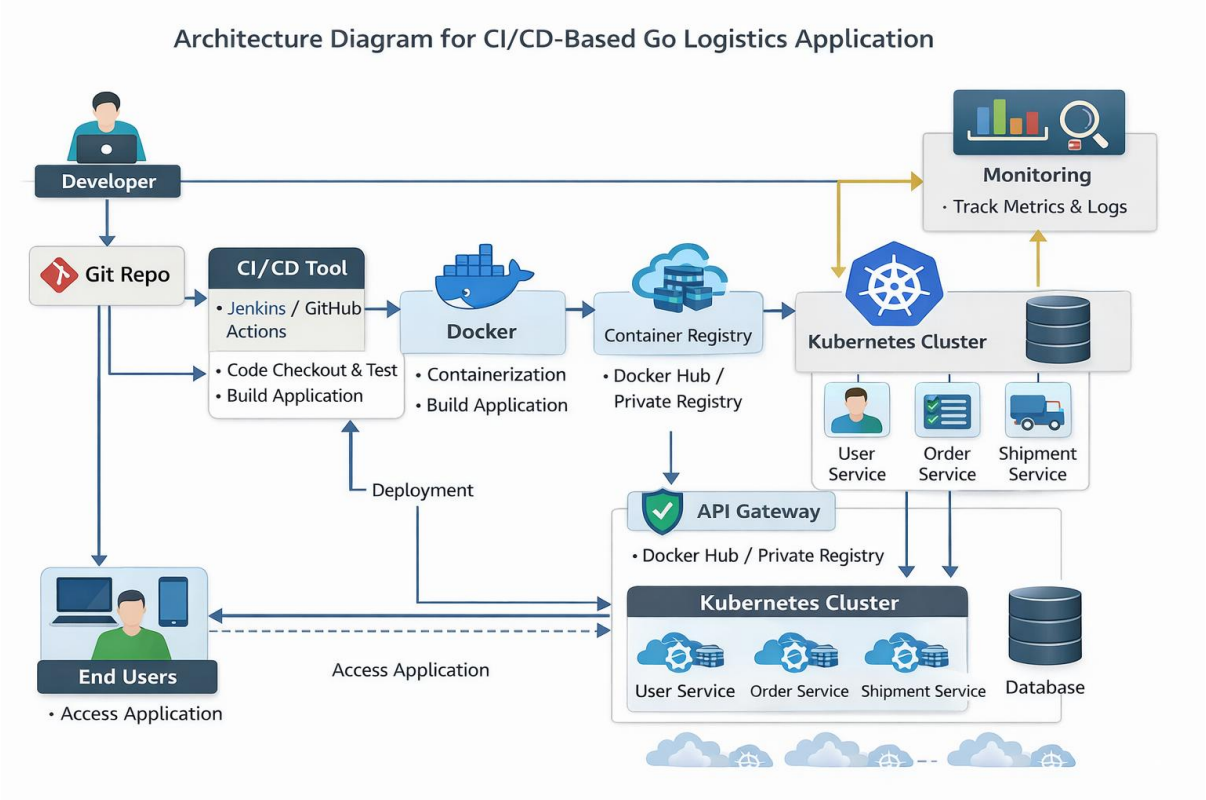

Fig: Architecture Diagram

2.2 Process Flow

The CI/CD workflow is divided into multiple stages:

1. **Code Commit Stage**

   - Developer pushes code to Git repository

   - Branching strategy:

   - Feature branch
   - Development branch
   - Main/production branch

2. **Continuous Integration (CI) Stage**

   - Triggered automatically on code push:
   - Code checkout
   - Dependency installation
   - Static code analysis (optional)
   - Unit testing execution
   - Build verification
   - If any step fails → pipeline stops

3. **Build & Containerization Stage**

- Go application is compiled
- Docker image is created using Dockerfile
- Image tagged with version (e.g., v1.0, commit ID)

4. **Artifact Storage**

- Docker image pushed to container registry
- Ensures version control of builds

5. **Continuous Deployment (CD) Stage**

- Triggered after successful build:
- Kubernetes deployment updated
- Rolling updates applied
- Pods recreated with new version

6. **6. Monitoring & Feedback**

- Logs generated
- Alerts for failures
- Feedback loop to developers

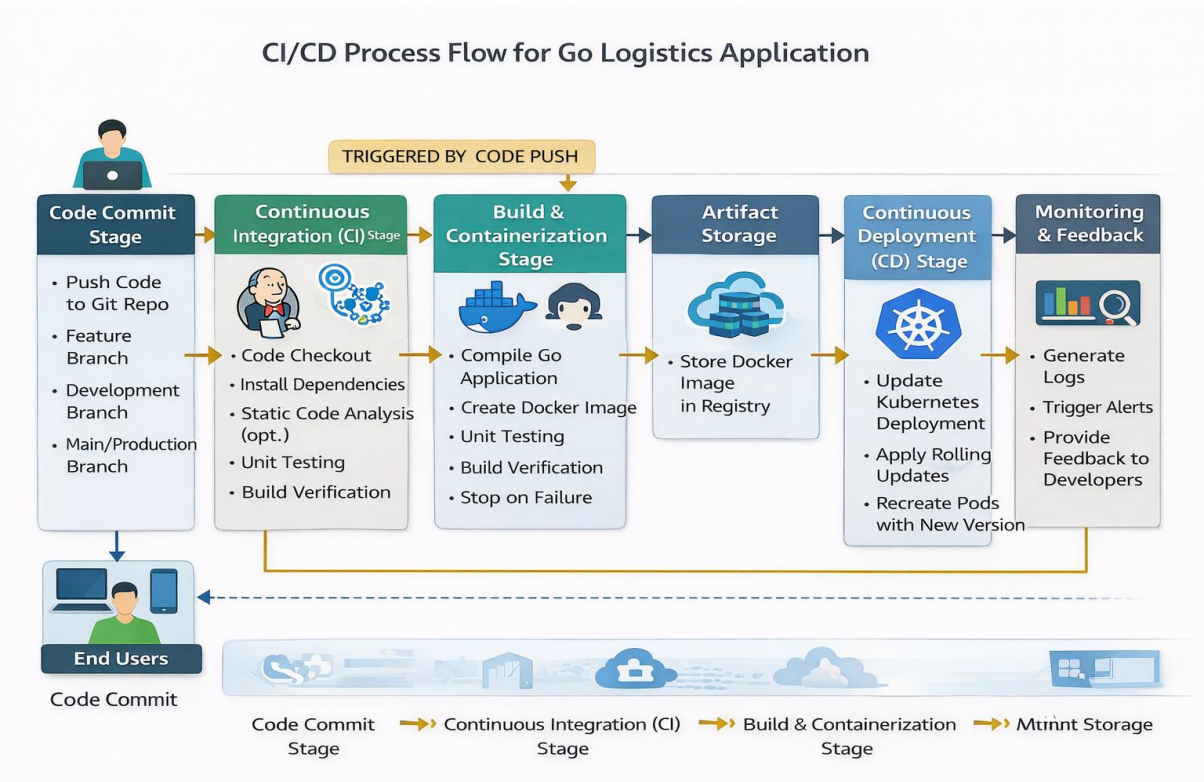

Fig: Process Fow Diagram

2.3 Information Flow

Detailed Flow:

- Developer → pushes code → Git
- Git → triggers webhook → CI tool
- CI tool → fetches code → executes pipeline
- Pipeline → builds Docker image
- Image → pushed to registry
- Kubernetes → pulls image → deploys
- Users → interact with deployed application

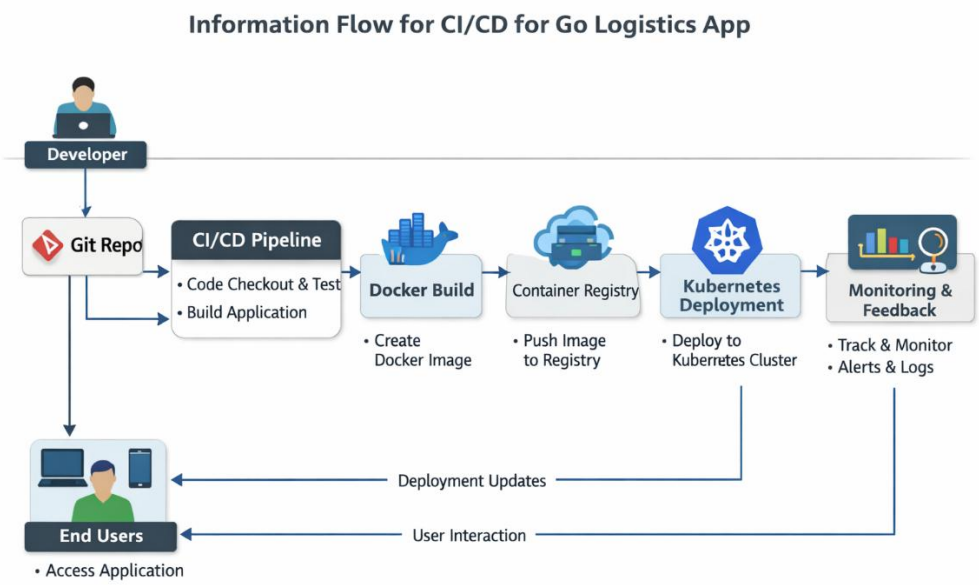

Fig: Information Flow

2.4 Components Design

1. Git Repository

- Stores application source code
- Maintains version history
- Enables collaboration

2. CI Tool (Jenkins / GitHub Actions)

- Automates:

- Build process
- Testing
- Deployment triggers
- Uses pipeline scripts

3. Docker

- Packages application into containers
- Eliminates environment dependency issues
- Ensures "build once, run anywhere"

4. Container Registry

- Central storage for Docker images

- Enables image versioning and rollback

5. Kubernetes Cluster

- Manages container deployment
- Handles:
- Load balancing
- Auto-scaling
- Self-healing

6. Load Balancer

- Distributes incoming traffic
- Ensures high availability

7. Monitoring Tools

- Track system performance
- Detect failures
- Provide logs and metrics.

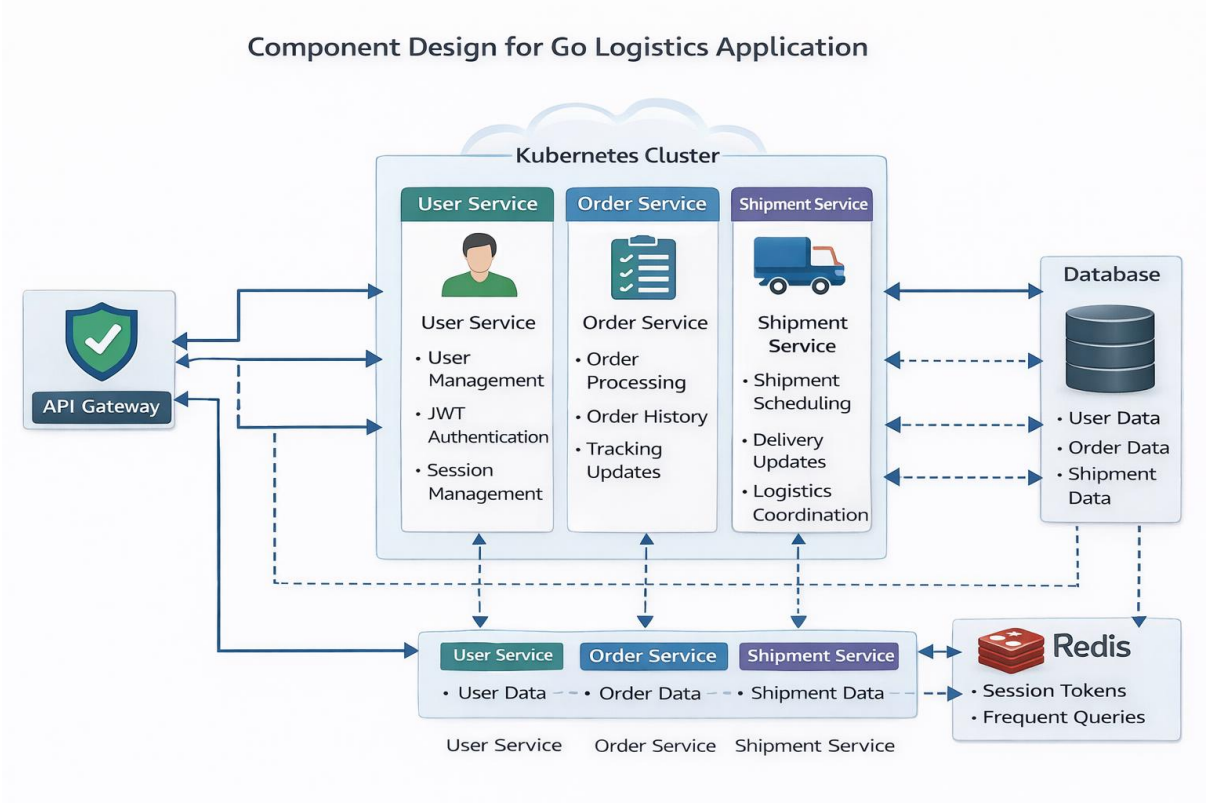

Fig: Component Design Diagram

2.5 Key Design Considerations

- Scalability: Kubernetes auto-scaling
- Reliability: Failover and self-healing
- Automation: Minimal manual intervention
- Consistency: Docker containers
- Security: Secrets management
- Maintainability: Modular design.

3. Data Design

3.1 Data Model

The application includes structured data entities

Entity	Description
User	Stores user details
Order	Contains order information
Shipment	Tracks shipment
Delivery	Delivery status

Relationships:

- One user can have multiple orders
- Each order has one shipment

3.2 Data Access Mechanism

- REST APIs for client-server interaction
- Database access via:
  - SQL / NoSQL queries
  - Secure authentication for access.

3.3 Data Retention Policies

- Application logs stored temporarily
- Old Docker images removed periodically
- Database backups scheduled
- Audit logs retained for complianceta privacy as no permanent copies of the embedded data remain after the session ends.

3.4 Data Migration

Data migration ensures smooth updates when system changes occur.

1. Schema Migration

- Database schema changes managed using:
- Migration scripts (version-controlled)
- Example:

- Adding new column
- Updating table structure

2. Version Control

- Each database change is:
- Tracked in Git
- Applied in sequence

3. Deployment Integration

- Migration scripts run automatically during deployment:
- Before application starts
- Ensures compatibility with new code

4. Rollback Strategy

- If deployment fails:
- Revert to previous schema version
- Restore database from backup

5. Zero Downtime Migration (Advanced)

- Use backward-compatible changes:
- Add fields instead of modifying directly
- Avoid service interruption

6. Data Consistency

- Ensure:
- No data loss
- No duplication
- Use transactions where required.

4. Interfaces

Interfaces define how different components of the system communicate with each other. In this CI/CD-based logistics application, interfaces ensure smooth interaction between the application, pipeline tools, and external services.

4.1 Internal Interfaces

Internal interfaces handle communication within the system components.

- **Service-to-Service Communication:**
  The application modules (Order, Shipment, User) communicate using REST APIs over HTTP/HTTPS with JSON format.

- **API Gateway:**
  Acts as a central entry point, routing requests to appropriate services and handling authentication and logging.
- **CI/CD Pipeline Communication:**
  Different stages (build, test, deploy) pass data internally. For example:
  - CI stage sends tested code to build stage
  - Build stage sends Docker image to deployment stage
- **Logging & Monitoring:**
  Application and pipeline logs are sent to monitoring tools for tracking performance and failures.

4.2 External Interfaces

External interfaces connect the system with third-party tools.

- **Git Repository (GitHub):**
  Used for source code management. Code commits trigger the CI/CD pipeline using webhooks.
- **CI Tool (Jenkins / GitHub Actions):**
  Executes automated workflows such as testing, building, and deployment.
- **Docker Registry:**
  Stores container images. CI pushes images, and Kubernetes pulls them during deployment.
- **Kubernetes API:**
  Handles deployment, scaling, and management of containers.

- **Database Interface:**
  Backend interacts with the database using queries for storing and retrieving application data.

4.3 User Interface

- **Web/API Interface:**
  Users interact with the system through a web application or APIs.
- **Functionalities include:**
  - User login and authentication
  - Order placement
  - Shipment tracking
- **Admin Interface:**
  Used for managing system operations like monitoring and order handling.

4.4 Communication Standards

- Protocol: HTTP/HTTPS

- Data format: JSON
- Uses RESTful APIs
- Standard HTTP status codes for responses

4.5 Design Considerations

- Secure communication using HTTPS
- Loose coupling between components
- Scalable and reliable interfaces
- Proper error handling and API versioning

5. State and Session Management

The system follows a stateless architecture where no session data is stored on the server, ensuring scalability and fault tolerance in a distributed environment like Kubernetes. User sessions are managed using JWT (JSON Web Tokens), where after successful login, a token containing user information is generated and sent to the client. This token is included in subsequent requests for authentication and authorization, eliminating the need for server-side session storage. Each request is independently processed, allowing any container or pod to handle it efficiently. To maintain security, tokens have an expiration time and are transmitted over HTTPS. This approach improves performance, simplifies load balancing, and supports seamless horizontal scaling of the application.

6. Caching

The system uses caching to improve performance and ensure faster response times in the Go-based logistics application deployed via the CI/CD pipeline. Frequently accessed data such as shipment status, order details, and user information is stored temporarily using an in-memory caching system like Redis. This reduces repeated database queries and minimizes latency for end users. Since the application is containerized and deployed on Kubernetes, the caching layer supports distributed access across multiple pods, ensuring consistency and scalability. Additionally, caching helps optimize resource utilization during high traffic, making the system more efficient and reliable in a continuously deployed environment.

7. Non-Functional Requirements

7.1 Security Aspects

- HTTPS for secure communication
- Authentication using JWT
- Role-based access control
- Secure storage of secrets:
- Environment variables
- Kubernetes Secrets
- Image vulnerability scanning

7.2 Performance Aspects

- Fast pipeline execution
- Reduced build time using caching
- Kubernetes auto-scaling
- Load balancing
- Optimized container size

8. References

- Go Programming Language Documentation
- Docker Official Docs
- Kubernetes Official Docs
- Jenkins / GitHub Actions Docs
- DevOps Best Practices